

1 目的

本実験では、順序回路 (10 進数カウンタ) を HDL で設計し、シュミレーションを行い、階層設計を行ってからモジュールごとの設計を行うことにより、FPGA において大規模な回路構築する手法を学ぶこと、そしてシュミレーションを行うことによりデバッグ作業を簡単化する方法について学ぶ。また、学んだ手法を活かして、課題より拡張させた機能の実装を行う。

2 原理

本実験で使用する GPGA, HDL, Verilog について説明する。

2.1 階層設計

大規模な回路を設計する際に、回路全体をモジュール単位に分割させ、モジュールごとのつながりをブロック図として表現する。各モジュールごとでの入出力を設計することにより、複雑な回路を簡単に設計することができる。また、モジュールの内部は階層構造に関係ないため、モジュールごとに独立して設計を行うことができる。

2.2 同期/非同期

同期回路とは、クロック信号に同期して動作する回路であり、クロックの立ち上がり/立下りに合わせて動作を行う。一方非同期とはクロックに依存しない回路であり、入力信号の立ち上がり/立下りに応じて動作を行う。

2.3 分周器

分周器とは、入力信号に対して一定の比率で出力信号を生成する回路である。クロック信号に同期したカウンタ回路とマジックナンバー (今回は分周する比率) を比較し、所定のカウント数に達したときに出力信号を切り替えることにより、分周を実現する。

2.4 ダイナミック点灯

7 セグメントの表示回路では、FPGA から出力電流 (8mA) を増幅させ、LED を明るくするためにソーストランジスタが用いられている。これらは一つの LED に対して一つのトランジスタ (実際にはダーリントントランジスタ構成になっていたり、クランプ回路が挟まっている) を配置しているため、本実験で使用する FPGA ボードでは 8ch のソーストランジスタアレイを用いている。これにより、 8mA を 20mA に増幅させている。これらの回路を 4 桁分用意すると、必要な IC の数が 4 個となり、回路がでかくなってしまう。そこでダイナミック点灯を用いる。7 セグメン

ト LED のコモンカソード側にシンクトランジスタアレイを配置している。これにより、各桁のコモンカソード側を GND にした桁にのみソーストランジスタアレイからの電流が流れる。この構成ではソーストランジスタアレイの各出力が 4 つの 7 セグメント LED のピンと同じように配線されている。そのため、ソーストランジスタアレイの入力とシンクトランジスタアレイの入力を同期させることにより、高速で桁の切り替えを行うことが出来る。人間の目では一定値以上の速度の切り替えを行うと同時に点滅しているように見える (残像効果) ため、4 桁分の 7 セグメント LED を少ない IC で点灯させることが出来る。

3 実験環境

表 1 に使用するソフトウェアとハードウェアについての実験環境を示す。

表 1: 実験環境

項目	内容
FPGA ボード	Gowin 社製 GW1NR-9
開発ソフトウェア	Gowin_V1.9.11.03 Education (64-bit)
シミュレーションソフトウェア	iverilog

4 実験方法

本実験で行った実験手順を示す。また、課題に対して追加した機能についても記載する。

4.1 ブロック図 (階層設計)

本実験で作成するものについて、仕様を決めてからブロック図を作成し、モジュール間の接続を明確にする。また、それぞれのモジュールについての動作についても仕様を決める。

4.2 HDL の作成

ブロック図をもとに HDL を作成する。本実験では Verilog を用いて HDL を作成する。

4.3 シミュレーション

作成した HDL が正しく動作するかをテストベンチを作成し、シミュレーションを行い確認する。

4.4 FPGA への書き込み

シミュレーションで正しく動作することが確認できたら，FPGA ボードに書き込みを行い，シミュレーションと同じ動きをするか確認する．

4.5 動作確認

FPGA ボードに書き込みを行った後，実際に動作確認を行う．

5 結果

実験方法を参照し，実験を行った．

5.1 ブロック図

図 1 に全加算器/全減算機の組み合わせ回路のブロック図を示す．また，各モジュールの仕様について示す（トップモジュールから見た動作仕様であり，内部の動作については実装した後に後述する）．

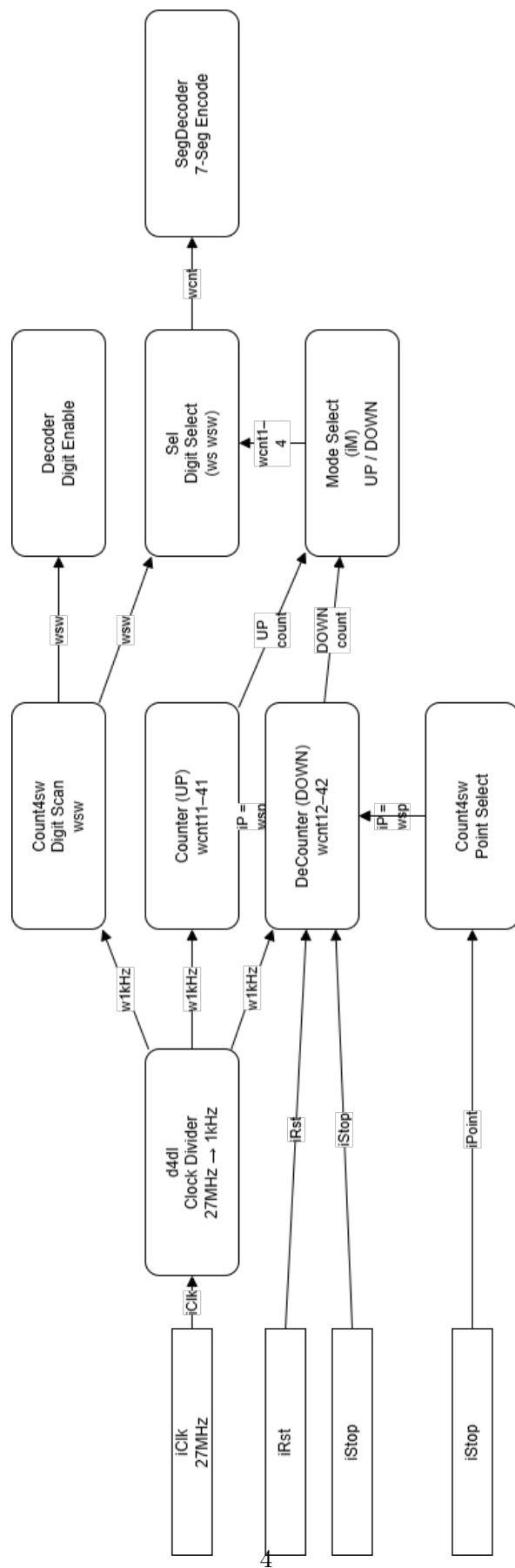


図 1: 全加算器/全減算機の組み合わせ回路のブロック図

5.1.1 分周器 (d4dl)

d4dl(DividerForDynamicLighting) は、システムクロックの $27m[Hz]$ を $1k[Hz]$ に分周する回路である。ダイナミック点灯と、7桁の10進カウンタの動作に使用している。入出力のピンの仕様を表2に示す。

表 2: 分周器 (d4dl) の入出力ピン仕様

ピン名	種類	説明
iClk	入力	システムクロック
oClk	出力	分周後クロック

5.1.2 4進カウンタ (Count4sw)

Count4sw(Count4switch) は、入力に応じて0から3までの4進カウントを行う回路である。今回は二個用いている。これは小数点表現をするために使用しており、 $1k[Hz]$ のクロック信号よりダイナミック点灯させている回路を制御しているため、今表示させている桁を wsw の信号で保持している。また、もう一つの回路では、スイッチの入力により4進カウンタを動作させており、小数点の位置を記憶している。小数点の位置と wsw の対応表を表3に示す。入出力のピンの仕様を表4に示す。

表 3: 小数点位置と wsw の対応表

wsw の値	小数点位置
00	0 桁目
01	1 桁目
10	2 桁目
11	3 桁目

表 4: 4進カウンタ (Count4sw) の入出力ピン仕様

ピン名	種類	説明
iClk	入力	クロック信号
oSw	出力	4進カウント値

5.1.3 7桁の10進カウンタ (Counter/DeCounter)

Counter/DeCounter は、7桁の10進カウンタである。Counter はカウントアップ、DeCounter はカウントダウンを行う。wsp の信号による小数点位置により、7桁のカウンタの値を1桁目、2桁目、3桁目、4桁目の出力とのつながりを変えている。入出力のピンの仕様を表5に示す。

表 5: 7桁の10進カウンタ (Counter/DeCounter) の入出力ピン仕様

ピン名	種類	説明
iClk	入力	クロック信号
iRst	入力	リセット信号
iStop	入力	カウント停止信号
iP	入力	小数点位置保持信号
oCnt1	出力	1桁目カウント値
oCnt2	出力	2桁目カウント値
oCnt3	出力	3桁目カウント値
oCnt4	出力	4桁目カウント値

5.1.4 4入力マルチプレクサ (Sel)

Sel(Selector) は、4入力マルチプレクサである。ダイナミック点灯を行うために、4桁の出力をwsw の信号により選択している。これにより、今表示している桁の値をソースランジスタアレイの入力として出力している。入出力のピンの仕様を表6に示す。

表 6: 4入力マルチプレクサ (Sel) の入出力ピン仕様

ピン名	種類	説明
iA	入力	1桁目に表示する値
iB	入力	2桁目に表示する値
iC	入力	3桁目に表示する値
iD	入力	4桁目に表示する値
iSw	入力	小数点位置保持信号
oOut	出力	選択された値

5.1.5 7セグメント用デコーダ (SegDecoder)

SegDecoder(SegmentDecoder) は、7セグメント LED 表示用のデコーダーである。4bit の2進数を7セグメント LED の点灯パターンに変換する回路である。入出力のピンの仕様を表7に示す。

表 7: 4 出力デコーダー(SegDecoder) の入出力ピン仕様

ピン名	種類	説明
iD	入力	デコードする前の 2 進数 (4bit)
iS	入力	小数点を表示するかの信号 (1bit)
oD	出力	7 セグメント LED の点灯パターン (8bit)

5.1.6 4 出力デコーダー(Decoder)

Decoder は, 2bit の 2 進数を 4bit の 1hot 信号 (1 つの bit だけ 1 であり, 他の Bit は 0 の信号) に変換する回路である. これによりダイナミック点灯のシフトレジスタアレイに入力する信号を生成している. 入出力のピンの仕様を表 8 に示す.

表 8: 4 出力デコーダー(Decoder) の入出力ピン仕様

ピン名	種類	説明
iA	入力	デコードする前の 2 進数 (2bit)
oF	出力	1hot 信号 (4bit)

5.2 HDL の作成

トップモジュールから見た動作仕様にに基づき, 各モジュールの HDL を作成した. 各モジュールのプログラムとその説明を以下に示す.

5.2.1 分周器 (d4dl)

分周器 (d4dl) の HDL コードをリスト 1 に示す. 動作で順としては, 内部のレジスタでクロックの立ち上がりをカウントし, 所定 (13499) のカウント数に達したときに出力クロックを反転させ, レジスタをクリアする. このマジックナンバーは, 以下の式で求めることが出来る. 2 で割っている理由としては, 立ち上がりクロックのみで立ち上がりと立下りを出力させているためである (反転動作を行うため). また, -1 している理由としては, 0 でリセットを行っているため, 1 からではなく 0 からカウントをするためである.

$$\frac{\text{入力クロック周波数[Hz]}}{\text{出力クロック周波数[Hz]}} - 1 = \text{マジックナンバー} \quad (1)$$

$$\frac{27[\text{MHz}]}{2 \times 1[\text{kHz}]} - 1 = 13499 \quad (2)$$

Listing 1: 分周器 (d4dl) の HDL コード

```

1 module d4dl(iClk, oClk);
2     input iClk; /* 27[MHz] */
3     output oClk; /* 1[kHz] */
4     reg [14:0] dd = 15'd13499; //14ビットカウンタ
5     reg oClk = 1'b0;
6
7     always @(posedge iClk) begin
8         if(dd == 15'd13499) begin //13,500 - 1
9             oClk <= ~oClk; //出力クロックを反転
10            dd <= 15'd0; //カウンタリセット
11        end
12        else begin
13            dd <= dd + 15'd1; //カウンタインクリメント
14        end
15    end
16 endmodule

```

5.2.2 4進カウンタ (Count4sw)

4進カウンタ (Count4sw) の HDL コードをリスト 2 に示す。このプログラムでは、内部のレジスタをそのまま出力に使ったコードで、内部レジスタの値が $(3)_{10}$ になったときに内部レジスタの値を $(0)_{10}$ をすることにより、簡単な 4進カウンタを実現している。

Listing 2: 4進カウンタ (Count4sw) の HDL コード

```

1 module Count4sw(iClk, oSw);
2     input iClk;
3     output reg [1:0] oSw = 2'b00;
4
5     always @(posedge iClk) begin
6         if (oSw == 2'b11) begin
7             oSw <= 2'b00;
8         end
9         else begin
10            oSw <= oSw + 2'b01;
11        end
12    end
13 endmodule

```

5.2.3 7桁の 10進カウンタ (Counter/DeCounter)

7桁の 10進カウンタ (Counter/DeCounter) の HDL コードをリスト 3 に示す。このプログラムでは 7つの内部レジスタをそれぞれカウントアップさせるコードであり、クロックに同期してい

る. $iRst$ が 0 の場合, すべての桁を 0 にし (同期リセット), また $iStop$ が 1 のときにすべての桁のカウンタアップを一時的に停止させるコードとなっている. $iRst$ が 0 かつ, $iStop$ が 1 のときには, 内部のカウンタ回路が動作するようになっている. 内部のカウンタでは, 一番下の桁がクロック信号の立ち上がりと同期してカウンタアップし, 各桁が $(10)_{10}$ になったときに次の桁をひとつカウンタアップさせ, その桁は $(0)_{10}$ にリセットする動作を行っている. これにより, 7 桁の 10 進カウンタを実現している. また, wsp の信号により, 小数点位置に応じて各桁の出力を入れ替える動作を行っている.

Listing 3: 7 桁の 10 進カウンタ (Counter/DeCounter) の HDL コード

```

1  module Counter(iClk,iRst,iStop,iP,oCnt1,oCnt2,oCnt3,oCnt4);
2      input iClk;
3      input iRst;
4      input iStop;
5      input [1:0] iP;
6      output [3:0] oCnt1;
7      output [3:0] oCnt2;
8      output [3:0] oCnt3;
9      output [3:0] oCnt4;
10     reg [3:0] cnt1 = 4'd0;
11     reg [3:0] cnt2 = 4'd0;
12     reg [3:0] cnt3 = 4'd0;
13     reg [3:0] cnt4 = 4'd0;
14     reg [3:0] cnt5 = 4'd0;
15     reg [3:0] cnt6 = 4'd0;
16     reg [3:0] cnt7 = 4'd0;
17     reg [3:0] cnt8 = 4'd0;
18
19     assign oCnt1 = iP == 2'b00 ? cnt4 :
20                 iP == 2'b01 ? cnt3 :
21                 iP == 2'b10 ? cnt2 : cnt1;
22     assign oCnt2 = iP == 2'b00 ? cnt5 :
23                 iP == 2'b01 ? cnt4 :
24                 iP == 2'b10 ? cnt3 : cnt2;
25     assign oCnt3 = iP == 2'b00 ? cnt6 :
26                 iP == 2'b01 ? cnt5 :
27                 iP == 2'b10 ? cnt4 : cnt3;
28     assign oCnt4 = iP == 2'b00 ? cnt7 :
29                 iP == 2'b01 ? cnt6 :
30                 iP == 2'b10 ? cnt5 : cnt4;
31
32     always @(posedge iClk) begin
33         if (~iRst) begin
34             cnt1 <= 4'd0;
35             cnt2 <= 4'd0;

```

```

36         cnt3 <= 4'd0;
37         cnt4 <= 4'd0;
38         cnt5 <= 4'd0;
39         cnt6 <= 4'd0;
40         cnt7 <= 4'd0;
41         cnt8 <= 4'd0;
42     end
43     else if (iStop) begin
44         cnt1 <= cnt1;
45         cnt2 <= cnt2;
46         cnt3 <= cnt3;
47         cnt4 <= cnt4;
48         cnt5 <= cnt5;
49         cnt6 <= cnt6;
50         cnt7 <= cnt7;
51         cnt8 <= cnt8;
52     end else if (cnt1 == 4'd9) begin
53         cnt1 <= 4'd0;
54         if (cnt2 == 4'd9) begin
55             cnt2 <= 4'd0;
56             if (cnt3 == 4'd9) begin
57                 cnt3 <= 4'd0;
58                 if (cnt4 == 4'd9) begin
59                     cnt4 <= 4'd0;
60                     if (cnt5 == 4'd9) begin
61                         cnt5 <= 4'd0;
62                         if (cnt6 == 4'd9) begin
63                             cnt6 <= 4'd0;
64                             if (cnt7 == 4'd9) begin
65                                 cnt7 <= 4'd0;
66                                 if (cnt8 == 4'd9) begin
67                                     cnt8 <= 4'd0;
68                                 end
69                             else begin
70                                 cnt8 <= cnt8 + 4'd1;
71                             end
72                         end
73                     else begin
74                         cnt7 <= cnt7 + 4'd1;
75                     end
76                 end
77             else begin
78                 cnt6 <= cnt6 + 4'd1;
79             end
80         end

```

```

81             else begin
82                 cnt5 <=cnt5 + 4'd1;
83             end
84         end
85         else begin
86             cnt4 <=cnt4 + 4'd1;
87         end
88     end
89     else begin
90         cnt3 <=cnt3 + 4'd1;
91     end
92 end
93 else begin
94     cnt2 <=cnt2 + 4'd1;
95 end
96 end
97 else begin
98     cnt1 <=cnt1 + 4'd1;
99 end
100 end
101
102 endmodule
103
104 module DeCounter(iClk,iRst,iStop,iP,oCnt1,oCnt2,oCnt3,oCnt4);
105     input iClk;
106     input iRst;
107     input iStop;
108     input [1:0] iP;
109     output [3:0] oCnt1;
110     output [3:0] oCnt2;
111     output [3:0] oCnt3;
112     output [3:0] oCnt4;
113     reg [3:0] cnt1 = 4'd0;
114     reg [3:0] cnt2 = 4'd0;
115     reg [3:0] cnt3 = 4'd0;
116     reg [3:0] cnt4 = 4'd0;
117     reg [3:0] cnt5 = 4'd0;
118     reg [3:0] cnt6 = 4'd0;
119     reg [3:0] cnt7 = 4'd0;
120     reg [3:0] cnt8 = 4'd0;
121
122     assign oCnt1 = iP == 2'b00 ? cnt4 :
123                 iP == 2'b01 ? cnt3 :
124                 iP == 2'b10 ? cnt2 : cnt1;
125     assign oCnt2 = iP == 2'b00 ? cnt5 :

```

```

126             iP == 2'b01 ? cnt4 :
127             iP == 2'b10 ? cnt3 : cnt2;
128     assign oCnt3 = iP == 2'b00 ? cnt6 :
129             iP == 2'b01 ? cnt5 :
130             iP == 2'b10 ? cnt4 : cnt3;
131     assign oCnt4 = iP == 2'b00 ? cnt7 :
132             iP == 2'b01 ? cnt6 :
133             iP == 2'b10 ? cnt5 : cnt4;
134
135     always @(posedge iClk) begin
136         if (~iRst) begin
137             cnt1 <= 4'd0;
138             cnt2 <= 4'd0;
139             cnt3 <= 4'd0;
140             cnt4 <= 4'd0;
141             cnt5 <= 4'd0;
142             cnt6 <= 4'd0;
143             cnt7 <= 4'd0;
144             cnt8 <= 4'd0;
145         end
146         else if (iStop) begin
147             cnt1 <= cnt1;
148             cnt2 <= cnt2;
149             cnt3 <= cnt3;
150             cnt4 <= cnt4;
151             cnt5 <= cnt5;
152             cnt6 <= cnt6;
153             cnt7 <= cnt7;
154             cnt8 <= cnt8;
155         end else if (cnt1 == 4'd0) begin
156             cnt1 <= 4'd9;
157             if (cnt2 == 4'd0) begin
158                 cnt2 <= 4'd9;
159                 if (cnt3 == 4'd0) begin
160                     cnt3 <= 4'd9;
161                     if (cnt4 == 4'd0) begin
162                         cnt4 <= 4'd9;
163                         if (cnt5 == 4'd0) begin
164                             cnt5 <= 4'd9;
165                             if (cnt6 == 4'd0) begin
166                                 cnt6 <= 4'd9;
167                                 if (cnt7 == 4'd0) begin
168                                     cnt7 <= 4'd9;
169                                     if (cnt8 == 4'd0) begin
170                                         cnt8 <= 4'd9;

```

```

171                                     end
172                                     else begin
173                                         cnt8 <=cnt8 - 4'd1;
174                                     end
175                                 end
176                                 else begin
177                                     cnt7 <=cnt7 - 4'd1;
178                                 end
179                             end
180                             else begin
181                                 cnt6 <=cnt6 - 4'd1;
182                             end
183                         end
184                         else begin
185                             cnt5 <=cnt5 - 4'd1;
186                         end
187                     end
188                     else begin
189                         cnt4 <=cnt4 - 4'd1;
190                     end
191                 end
192                 else begin
193                     cnt3 <=cnt3 - 4'd1;
194                 end
195             end
196             else begin
197                 cnt2 <=cnt2 - 4'd1;
198             end
199         end
200         else begin
201             cnt1 <=cnt1 - 4'd1;
202         end
203     end
204
205 endmodule

```

5.2.4 4入力マルチプレクサ (Sel)

4入力マルチプレクサ (Sel) のHDLコードをリスト4に示す。このプログラムでは、入力された4つの信号の中から、iSwの信号に応じて1つを選択して出力するコードとなっている。Self関数において、case文を用いて、oOutと入力された信号のつながりを変更している。

Listing 4: 4 入力マルチプレクサ (Sel) の HDL コード

```

1 module Sel(iA,iB,iC,iD,iSw,oOut);
2   input [3:0]iA;
3   input [3:0]iB;
4   input [3:0]iC;
5   input [3:0]iD;
6   input [1:0]iSw;
7   output [3:0]oOut;
8   assign oOut = SelF(iA,iB,iC,iD,iSw);
9   function [3:0] SelF;
10    input [3:0] iA;
11    input [3:0] iB;
12    input [3:0] iC;
13    input [3:0] iD;
14    input [1:0] iSw;
15  begin
16    case(iSw)
17      2'b00: SelF = iA;
18      2'b01: SelF = iB;
19      2'b10: SelF = iC;
20      2'b11: SelF = iD;
21      default: SelF=4'bXXXX;
22    endcase
23  end
24  endfunction
25 endmodule

```

5.2.5 7 セグメント用デコーダ (SegDecoder)

7 セグメント用デコーダ (SegDecoder) の HDL コードをリスト 5 に示す。このプログラムでは、入力された 4bit の 2 進数を 7 セグメント LED の点灯パターンに変換するコードとなっている。また、iS の信号が 1 のときに小数点を点灯させる動作を行っている。

Listing 5: 7 セグメント用デコーダ (SegDecoder) の HDL コード

```

1 module SegDecoder(iD,iS,oD);
2   input [3:0] iD; //0~9の4ビット入力
3   input iS;
4   output [7:0] oD;
5   assign oD = ~DecF(iD);
6   function [7:0] DecF;
7     input [3:0] iD;
8     begin
9       case(iD)
10        4'b0000: DecF = {iS,7'b1000000}; // 0

```

```

11      4'b0001: DecF = {iS,7'b1111001}; // 1
12      4'b0010: DecF = {iS,7'b0100100}; // 2
13      4'b0011: DecF = {iS,7'b0110000}; // 3
14      4'b0100: DecF = {iS,7'b0011001}; // 4
15      4'b0101: DecF = {iS,7'b0010010}; // 5
16      4'b0110: DecF = {iS,7'b0000010}; // 6
17      4'b0111: DecF = {iS,7'b1111000}; // 7
18      4'b1000: DecF = {iS,7'b0000000}; // 8
19      4'b1001: DecF = {iS,7'b0010000}; // 9
20      default: DecF = 8'b11111111; // 消灯
21  endcase
22  end
23 endfunction
24
25 endmodule

```

5.2.6 4 出力デコーダー(Decoder)

4 出力デコーダー(Decoder) の HDL コードをリスト 6 に示す. このプログラムでは, 入力された 2bit の 2 進数を 4bit の 1hot 信号に変換するコードとなっている.

Listing 6: 4 出力デコーダー(Decoder) の HDL コード

```

1  module Decoder(iA, oF);
2      input  [1:0] iA;
3      output [3:0] oF;
4
5      function [3:0] Dec;
6          input [1:0] iA;
7          begin
8              case(iA)
9                  2'b00: Dec = 4'b0001;
10                 2'b01: Dec = 4'b0010;
11                 2'b10: Dec = 4'b0100;
12                 2'b11: Dec = 4'b1000;
13                 default:Dec = 4'bXXXX;
14             endcase
15         end
16     endfunction
17     assign oF = Dec(iA);
18 endmodule

```

5.3 トップモジュール (MAIN)

トップモジュール (MAIN) の HDL コードをリスト 7 に示す. このプログラムでは, 各モジュールをインスタンス化し, 接続を行っている. また, トップモジュールでは, 全加算器/全減算機の組み合わせ回路の動作仕様にに基づき, スイッチの入力によりカウントアップ/カウントダウンの切り替えを行っている.

Listing 7: トップモジュール (MAIN) の HDL コード

```
1 module MAIN(iClk,iPoint,iM,iStop,iRst,oD,oD1,oD2,oD3,oD4);
2     input iClk; // 27MHz クロック入力
3     input iPoint;
4     input iM;
5     input iRst;
6     input iStop;
7     output [7:0] oD;
8     output oD1;
9     output oD2;
10    output oD3;
11    output oD4;
12
13
14    // wire w1Hz;
15    wire w1kHz;
16    // wire wd1kHz;`
17    wire [3:0] wcnt1;
18    wire [3:0] wcnt2;
19    wire [3:0] wcnt3;
20    wire [3:0] wcnt4;
21    wire [3:0] wcnt11;
22    wire [3:0] wcnt21;
23    wire [3:0] wcnt31;
24    wire [3:0] wcnt41;
25    wire [3:0] wcnt12;
26    wire [3:0] wcnt22;
27    wire [3:0] wcnt32;
28    wire [3:0] wcnt42;
29
30    wire [3:0] wcnt;
31    wire [1:0] wsw;
32    wire [1:0] wsp;
33    wire ws;
34    assign ws = (wsw == wsp )? 1'b0 : 1'b1;
35
```



```

36    // d4c u1(.iClk(iClk), .oClk(w1Hz));
37    d4dl u2(.iClk(iClk), .oClk(w1kHz));
38    Count4sw usw(.iClk(w1kHz), .oSw(wsw));
39    Count4sw usw2(.iClk(iPoint), .oSw(wsp));
40    Counter u3(.iClk(w1kHz), .iRst(iRst), .iStop(~iStop), .iP(wsp), .oCnt1(wcnt11), .
        oCnt2(wcnt21), .oCnt3(wcnt31), .oCnt4(wcnt41));
41    DeCounter u3d(.iClk(w1kHz), .iRst(iRst), .iStop(~iStop), .iP(wsp), .oCnt1(wcnt12),
        .oCnt2(wcnt22), .oCnt3(wcnt32), .oCnt4(wcnt42));
42    assign wcnt1 = iM == 1'b0 ? wcnt11 : wcnt12;
43    assign wcnt2 = iM == 1'b0 ? wcnt21 : wcnt22;
44    assign wcnt3 = iM == 1'b0 ? wcnt31 : wcnt32;
45    assign wcnt4 = iM == 1'b0 ? wcnt41 : wcnt42;
46    Sel u4(.iA(wcnt1), .iB(wcnt2), .iC(wcnt3), .iD(wcnt4), .iSw(wsw), .oOut(wcnt));
47    SegDecoder u5(.iD(wcnt), .iS(ws), .oD(oD));
48    Decoder u6(.iA(wsw), .oF({oD4,oD3,oD2,oD1}));
49    endmodule

```

5.4 シミュレーション

作成した HDL が正しく動作するかをテストベンチを作成し、シミュレーションを行い確認した。今回のコードでは初期状態では、w1kHz のクロック信号が 1k 回分立ち上がった後に、最初の桁が繰り上がりされる。また、w1kHz も iClk を 13500 回分立ち上がった後に出力が反転する。そのため simulation に非常に時間がかかるため、最初に iPoint の信号を (11) に設定し、小数点位置を 3 桁目に固定している。また、すべてのモジュールのクロック入力で w1kHz を使用しているところは iClk に置き換えてテストベンチを動作させた。そして、加算/減算を行えるため、二種類の結果を示す。加算の結果をシミュレーション 1、減算の結果をシミュレーション 2 として示す。テストベンチとして作成したコードをリスト 8 に示す。

Listing 8: 全加算器/全減算機の組み合わせ回路のテストベンチコード

```

1
2  `timescale 10ns / 1ps
3  module MAINTTEST;
4
5  reg iClk = 0;
6  reg iRst = 0;
7  reg iPoint = 0;
8  reg iM = 0;
9  reg iStop = 0;
10
11 wire [7:0] oD;
12 wire oD1;
13 wire oD2;

```

```

14 wire oD3;
15 wire oD4;
16
17 MAIN main_inst(
18     iClk,
19     iPoint,
20     iM,
21     iStop,
22     iRst,
23     oD,
24     oD1,
25     oD2,
26     oD3,
27     oD4
28 );
29
30 // クロック生成 (500MHz)
31 always #1 iClk = ~iClk;
32
33 initial begin
34     // 初期化
35     iClk    = 0;
36     iRst    = 1;
37     iPoint  = 0;
38     iM      = 0;
39     iStop   = 0;
40
41     // 波形ダンプ
42     $dumpfile("main.vcd");
43     $dumpvars(0, MAINTTEST);
44     #1 iRst = 1;
45     //     #1 iM=1;
46     #1 iStop = 0;
47     #1 iPoint = 1;
48     #1 iPoint = 0;
49     #1 iPoint = 1;
50     #1 iPoint = 0;
51     #1 iPoint = 1;
52     #1 iPoint = 0;
53     #1 iStop = 1;
54     #1000;
55
56     $finish;
57 end
58

```

シミュレーション 1 の結果を図 2 に示す。シミュレーション 2 の結果を図 3 に示す。また、このシミュレーション結果にはテストベンチの下にあるトップモジュールの信号、wcnt1 4 を含めており、10 進数の可読性をあげている。そしてこの実験結果の波形解析を以下で行う。

5.4.1 シミュレーション 1

シミュレーション 1 では、加算動作を行うテストベンチであり、iStop が 1 になった次のクロックの立ち上がりで wcnt1 = 1 になることから、カウントアップが開始されていることが読める。そして、iClk と同期して 1 ずつカウントアップされ、9 の次に 0 になり、wcnt2 が 1 になることから桁上がりもうまくいっていることがわかる。次に、oD1 4 と oD の出力を比べると、iStop が 1 になった次のクロックから 10 クロックまでの間、oD1=1 ではないとき oD は 0 を表しており、oD1=1 のときに oD は wcnt1 の値を 7 セグメント LED 用にデコードした値を示しているはずである。出力結果を確認すると、wcnt1 が 3 のとき、oD1 が 1 のため確認してみると、oD は 7 セグメント LED 用にデコードされた値である $(0b01001111)_2$ を示しており、正しい動作をしていることがわかる。よって、加算動作においての桁上がりと、7 セグメント LED 用のデコードが正しく動作していることが分かった。

5.4.2 シミュレーション 2

シミュレーション 2 では、減算動作を行うテストベンチであり、iM = 1 に設定されている。シミュレーション 1 では 7 セグメント LED 用のデコード回路が正しく動作することを確認できたため、カウントダウン動作のみを確認する。iStop が 1 になった次のクロックの立ち上がりで wcnt1 = 9 になることから、カウントダウンが開始されていることが読める。そして、iClk と同期して 1 ずつカウントダウンされ、0 の次に 9 になり、wcnt2 が 8 になることから桁下がりもうまくいっていることがわかる。よって、減算動作においての桁下がりが正しく動作していることが分かった。

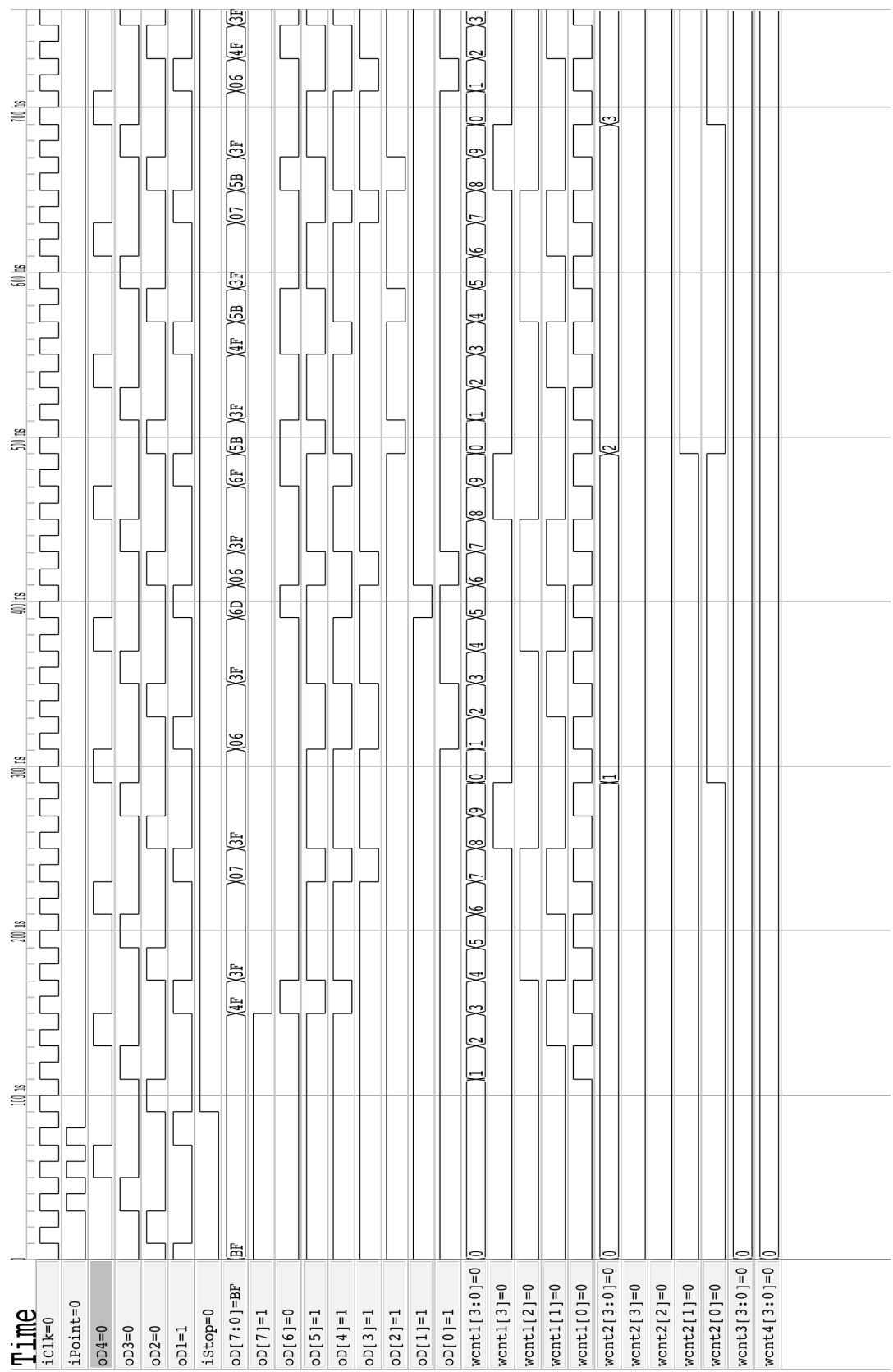


図 2: シミュレーション結果 1

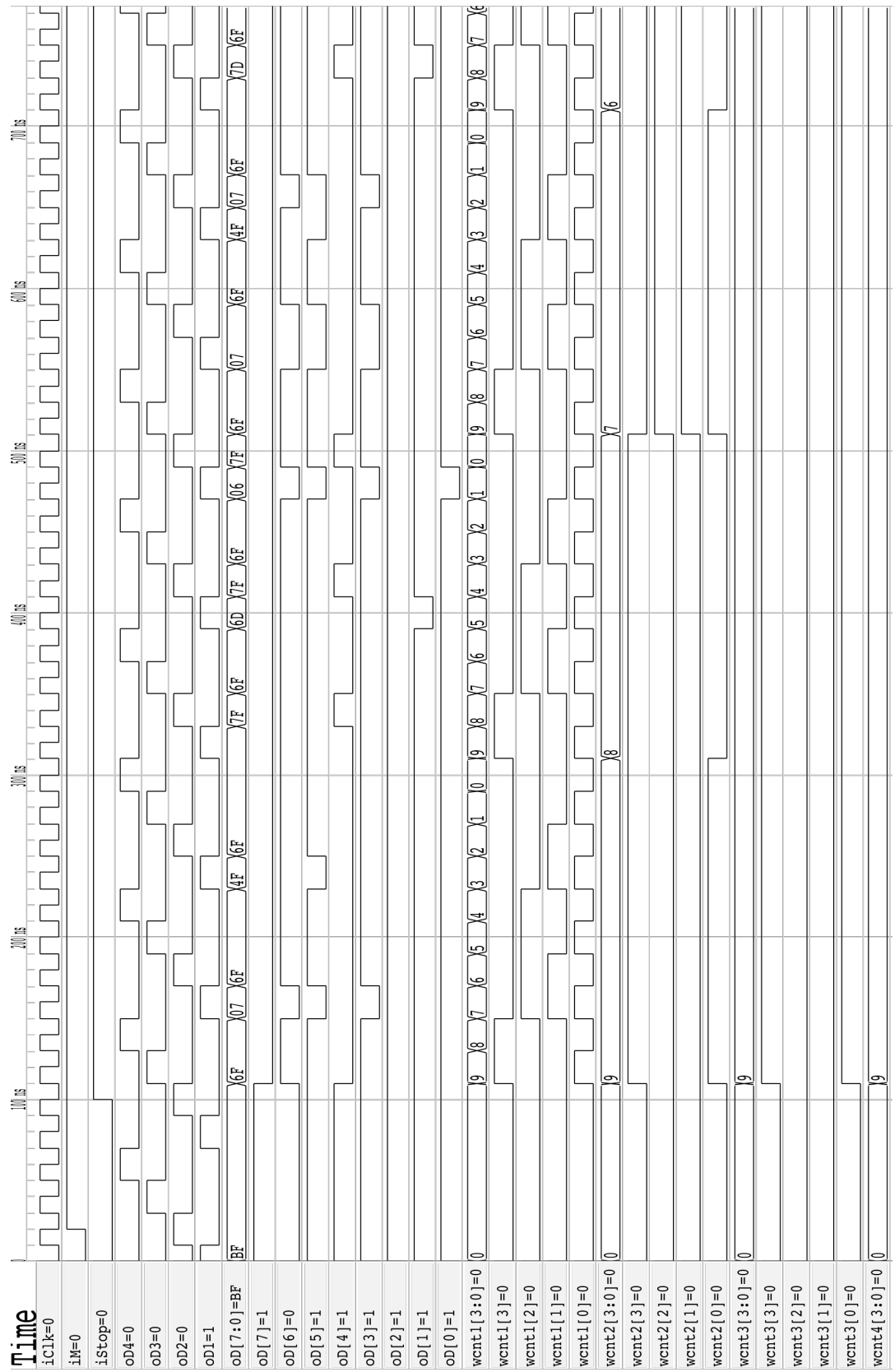


図 3: シミュレーション結果 2

5.5 FPGA への書き込み

シミュレーションで正しく動作することが確認できたため、FPGA ボードに書き込みを行った。また、設定を行ったピン番号をリスト 9 に示し、各信号との対応表を表 9 に示す。

Listing 9: FPGA ピン設定コード

```
1 //Copyright (C)2014-2025 Gowin Semiconductor Corporation.
2 //All rights reserved.
3 //File Title: Physical Constraints file
4 //Tool Version: V1.9.11.03 Education
5 //Part Number: GW1NR-LV9QN88PC6/I5
6 //Device: GW1NR-9
7 //Device Version: C
8 //Created Time: Wed 12 10 15:35:51 2025
9
10 IO_LOC "oD4" 86;
11 IO_PORT "oD4" IO_TYPE=LVCMS18 PULL_MODE=UP DRIVE=8 BANK_VCCIO=1.8;
12 IO_LOC "oD3" 85;
13 IO_PORT "oD3" IO_TYPE=LVCMS18 PULL_MODE=UP DRIVE=8 BANK_VCCIO=1.8;
14 IO_LOC "oD2" 84;
15 IO_PORT "oD2" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
16 IO_LOC "oD1" 83;
17 IO_PORT "oD1" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
18 IO_LOC "oD[7]" 49;
19 IO_PORT "oD[7]" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
20 IO_LOC "oD[6]" 48;
21 IO_PORT "oD[6]" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
22 IO_LOC "oD[5]" 70;
23 IO_PORT "oD[5]" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
24 IO_LOC "oD[4]" 71;
25 IO_PORT "oD[4]" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
26 IO_LOC "oD[3]" 72;
27 IO_PORT "oD[3]" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
28 IO_LOC "oD[2]" 73;
29 IO_PORT "oD[2]" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
30 IO_LOC "oD[1]" 74;
31 IO_PORT "oD[1]" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
32 IO_LOC "oD[0]" 75;
33 IO_PORT "oD[0]" IO_TYPE=LVCMS18 PULL_MODE=NONE DRIVE=8 BANK_VCCIO=1.8;
34 IO_LOC "iRst" 4;
35 IO_PORT "iRst" IO_TYPE=LVCMS18 PULL_MODE=NONE BANK_VCCIO=1.8;
36 IO_LOC "iStop" 3;
37 IO_PORT "iStop" IO_TYPE=LVCMS18 PULL_MODE=UP BANK_VCCIO=1.8;
38 IO_LOC "iM" 31;
```

```

39 IO_PORT "iM" IO_TYPE=LVC MOS18 PULL_MODE=UP BANK_VCCIO=1.8;
40 IO_LOC "ipoint" 32;
41 IO_PORT "ipoint" IO_TYPE=LVC MOS18 PULL_MODE=UP BANK_VCCIO=1.8;
42 IO_LOC "iClk" 52;
43 IO_PORT "iClk" IO_TYPE=LVC MOS18 PULL_MODE=NONE BANK_VCCIO=1.8;

```

表 9: 各信号との対応表

信号名	ピン番号 IOTYPE	PULL MODE
oD4-oD1	83-86 LVC MOS18	UP
oD[7]-oD[0]	49-75 LVC MOS18	NONE
iRst	4 LVC MOS18	NONE
iStop	3 LVC MOS18	UP
iM	31 LVC MOS18	UP
ipoint	32 LVC MOS18	UP
iClk	52 LVC MOS18	NONE

5.6 動作確認

FPGA ボードに書き込みを行った後、実際に動作確認を行った。画像 4 では、露光時間を 1/50 秒に設定して撮影したものであり、ダイナミック点灯 (1/1000 秒) の 4 桁分の時間よりも長いためすべての桁がはっきり見れることを表している。画像 5 では、露光時間を 1/6000 秒に設定して撮影したものであり、ダイナミック点灯 (1/1000 秒) よりも短いため、1 桁分しか見えないことを表している。この二つの画像より、ダイナミック点灯が正しく動作していることが分かる。また、操作機能として、iRst と iStop、また、iM、ipoint のスイッチを操作することにより、リセット、カウンタの停止/再開、加算/減算、小数点位置の変更が正しく動作していることを確認した。



図 4: FPGA ボードの動作確認画像 1(露光時間 1/50 秒)



図 5: FPGA ボードの動作確認画像 2(露光時間 1/6000 秒)

6 考察

本実験で得られた結果をもとに，達成目標ごとの達成度と考察を述べる．

6.1 順序回路の階層設計

本実験では，順序回路 (10 進数カウンタ) を HDL で設計し，階層設計を用いて大規模な回路を構築する手法を実践した．図 1 に示したブロック図のように，システム全体を分周器，カウンタ，

セレクタ、デコーダといった機能単位のモジュールに分割し、それぞれのモジュールに対して明確な入出力仕様を定義した。この手法により、各モジュールの動作を独立して設計・検証することができ、複雑な回路の実装を効率的に進めることができた。

特に、7桁の10進カウンタのように比較的複雑なモジュールにおいても、入出力ピンの仕様を明確にすることで、トップモジュールでの配線が容易になり、モジュール間の接続ミスを防ぐことができた。また、各モジュールを独立して設計することで、デバッグ作業も効率化され、問題が発生した際にも該当するモジュールに絞って原因を特定することが可能であった。

課題通りの設計を行った後に拡張機能を追加したため、モジュールの命名規則や信号名の統一が重要であることも学んだ。一貫した命名規則により、コードの可読性と保守性が向上し、拡張時の混乱を防ぐことができた。

6.2 HDL 設計とシミュレーション

Verilog HDL を用いた回路設計において、テストベンチを作成してシミュレーションを行うことの重要性を実感した。特に本実験では、FPGA への書き込み前にシミュレーションを行うことで、カウンタの桁上がり・桁下がり動作や、7セグメント LED のデコード処理が正しく動作することを確認できた。

シミュレーション 1 では加算動作における桁上がりの動作を確認し、`wcnt1` が 9 から 0 に変化する際に `wcnt2` が 1 増加することから、正しく繰り上がり処理が行われていることが確認できた。また、シミュレーション 2 では減算動作における桁下がり動作を確認し、`wcnt1` が 0 から 9 に変化する際に `wcnt2` が 1 減少することから、正しく繰り下がり処理が行われていることが確認できた。

このように、シミュレーションを活用することで、実機での動作確認前にロジックエラーを検出・修正することができ、開発効率を大幅に向上させることができた。シミュレーションによるデバッグ作業の単純化という目的は十分に達成できたと言える。

6.3 分周器の設計と動作

システムクロック 27MHz を 1kHz に分周する回路を設計し、実装した。分周比を決定するマジックナンバーの計算において、出力クロックの反転動作を考慮して入力周波数を 2 倍で割る必要があることや、0 からカウントを開始するため -1 する必要があることなど、実装上の細かい注意点を理解することができた。

この分周器は、ダイナミック点灯とカウンタ動作の両方に使用されており、システム全体の時間軸を制御する重要な役割を果たしている。実際の動作確認でも安定して 1kHz の信号が生成されており、設計通りの動作が実現できたことを確認した。

6.4 ダイナミック点灯の実装

7 セグメント LED のダイナミック点灯を実装し、4 桁の表示を実現した。画像 4 と画像 5 の比較から、1kHz の周期で各桁を順次点灯させることで、人間の目には 4 桁が同時に点灯しているように見えることが確認できた。

露光時間 1/50 秒の画像では 4 桁すべてがはっきりと見えるのに対し、露光時間 1/6000 秒の画像では 1 桁分しか見えないことから、実際に高速で桁の切り替えが行われていることが視覚的に確認できた。この手法により、ソースランジスタアレイを 1 組のみ使用して 4 桁の表示を実現でき、回路の小型化とコスト削減を達成できた。

ダイナミック点灯のメカニズムとして、ソースランジスタアレイとシンクランジスタアレイを組み合わせることで、少ない IC で複数桁の制御を実現できることを実践的に理解できた。

6.5 拡張機能の実装

課題で求められていた基本的な 10 進カウンタに加えて、以下の拡張機能を実装した。

6.5.1 加算/減算の切り替え機能

iM スイッチにより、カウントアップとカウントダウンを切り替える機能を実装した。シミュレーション結果から、両方の動作が正しく実装されていることが確認でき、実機での動作確認でも問題なく動作した。

6.5.2 小数点位置の可変機能

ipoint スイッチにより、小数点の表示位置を 0 桁目から 3 桁目まで変更できる機能を実装した。この機能は 4 進カウンタを用いて小数点位置を記憶し、カウンタモジュール内部で各桁の出力を入れ替えることで実現している。

6.5.3 カウント停止/再開機能

iStop スイッチにより、カウント動作を一時停止し、再開できる機能を実装した。この機能は同期回路として実装されており、クロックに同期してカウント動作の停止/再開が行われる。実機での動作確認でも正確に動作することが確認でき、ユーザーが任意のタイミングでカウント値を保持できるようになった。

6.6 同期設計の重要性

本実験では、すべての順序回路をクロックに同期させて設計した。同期設計を採用することで、タイミング解析が容易になり、回路の信頼性が向上することを実感した。特に FPGA のような大規模集積回路では、同期設計が回路の安定動作に不可欠であることが理解できた。

6.7 トランジスタの動作時間

本実験では、FPGA 上での設計・実装を行ったため、実際のトランジスタの動作時間に関する詳細な検討は行わなかった。しかし、ダイナミック点灯の実装においては、画像 6 のように、次の桁にわずかに一瞬だけ、前の桁の一部が 7 セグメント LED に表示されていることがわかる。これは、表示する桁を切り替える回路よりも 7 セグメントデコーダの出力回路のほうが回路が複雑であり、トランジスタの動作時間が長いために発生していると考えられる。この特性を考えた場合、より完璧な表示にさせるためには桁の表示を、1hot 信号にするのではなく、1hot 信号と 1hot 信号のクロックに同期して切り替えるところでわずかに全部の出力を 0 にするタイミングを設けることにより、前の桁の影響を防ぐことが出来ると考えられる。

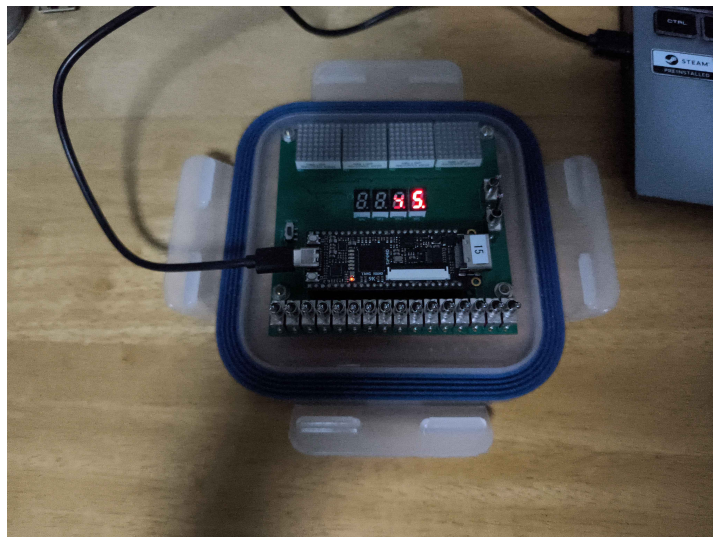


図 6: FPGA ボードの動作確認画像 3(露光時間 1/2000 秒)

6.8 今後の課題と改善点

本実験を通じて、いくつかの改善点や今後の課題が明らかになった。

6.8.1 シミュレーション時間の短縮

実際の分周比を用いたシミュレーションでは非常に長い時間がかかるため、今回はクロックを置き換えて検証を行った。今後は、シミュレーション用のパラメータを設定し、分周比を可変にできる設計にすることで、実機と同じ設定でのシミュレーションと、高速なシミュレーションの両方に対応できるようにすることが望ましい。

6.8.2 エラー処理の実装

現在の実装では、カウンタの上限値に達した場合の処理が明確でない。例えば 7 桁すべてが 9 になった後にさらにカウントアップした場合、オーバーフローが発生して 0 に戻るが、この状態を検出して警告を出す機能があると、より実用的なカウンタになると考えられる。

6.8.3 消費電力の最適化

ダイナミック点灯により回路の小型化は実現できたが、消費電力の観点からの最適化は行っていない。例えば、表示する桁数が少ない場合に不要な桁の点灯を停止したり、輝度を調整したりする機能を追加することで、さらなる省電力化が可能になると考えられる。

7 結論

本実験では、順序回路の階層設計手法を用いて 10 進数カウンタを FPGA 上に実装し、以下の成果を得た。

1. 階層設計により、複雑な回路を機能単位のモジュールに分割し、効率的に設計・実装できることを実証した。
2. Verilog HDL を用いた回路設計とシミュレーションにより、実機での動作確認前にロジックエラーを検出・修正できることを確認した。
3. ダイナミック点灯を用いた 4 桁の 7 セグメント LED 表示を実装し、回路の小型化を実現した。
4. 基本的なカウンタ機能に加えて、加算/減算の切り替え、小数点位置の可変、カウント停止/再開といった拡張機能を実装し、カウンタの汎用性を向上させた。
5. 同期設計の重要性を理解し、安定した回路動作を実現した。

これらの成果により、FPGA を用いた大規模回路の設計手法と、HDL による回路記述の基本を習得することができた。今後は、本実験で得た知識を基に、より複雑な順序回路や組み合わせ回路の設計に取り組んでいきたい。